GEbPDCM + LTPM

Run / Simulator Guide

Step-by-step instructions for running v1 (NS-3) and v2 (Python), and the live demo.

1. Prerequisites

- WSL2 with an Ubuntu distro (all commands below run inside WSL, not Windows PowerShell/cmd).
- ns-3.42 already downloaded and built at ~/ns-allinone-3.42/ns-3.42 (needed for v1 only; v2 never touches NS-3 after the topology has been generated once).
- Python 3 with numpy, pandas, matplotlib installed (pip3 install numpy pandas matplotlib).
- Both project folders present: ~/networks_project_v1_ns3 and ~/networks_project_v2_python.

Everything below is a plain bash command -- run it exactly as written from inside a WSL terminal.

2. Running v1 (Real NS-3)

v1's engine is ns3/gebpdcm.cc -- a real, packet-level NS-3 simulation. This is slower than v2 but produces genuine, physically grounded results.

2.1 Re-run the full sweep (4 schemes x 5 speeds, ~5 minutes)

```
cd ~/networks_project_v1_ns3
./ns3/run_sweep.sh
```

This rebuilds ns3/gebpdcm.cc if it has changed, then runs all 20 combinations (baseline / gebpdcm / gebpdcm_geo / gebpdcm_ltpm, each at 10/20/30/40/50 m/s), writing:

- data/topology.csv -- the 100 real node positions (regenerated fresh, same every run due to a fixed RNG seed)
- data/ns3_<scheme>_speed<N>.csv -- 20 files, one per scheme/speed combination, each holding delay_ms/throughput_mbps/pdr/packet_drop_ratio/energy_mj

You do NOT need to re-run this if you just want to regenerate the graphs/report from the already-saved data/ns3_*.csv files -- skip straight to 2.2.

2.2 Turn the NS-3 output into graphs + tables

```
cd ~/networks_project_v1_ns3
python3 run_all.py       # reads data/ns3_*.csv -> writes results/speed_sweep.csv, all_results.csv
python3 plot_results.py  # writes the 5 PNGs + results/comparison_table.csv
```

run_all.py also runs a secondary packet-size sweep via the Python reference simulator (src/simulator.py) -- this is the ONE place v1 still uses Python, since the packet-size sweep was never ported to NS-3 (the spec's own priority order cuts it first if short on time).

2.3 Optional: run just one scheme/speed by hand

```
cd ~/ns-allinone-3.42/ns-3.42
./ns3 run "gebpdcm --scheme=gebpdcm_ltpm --speed=30 --packetSize=1024 --simTime=100"
```

NOTE the syntax: target and args go in ONE quoted string, space-separated, with NO "--" before --scheme. ("./ns3 run \"gebpdcm -- --scheme=...\"" -- with a literal "--" -- fails on this ns-3 version with "Invalid command-line argument: --", since that token gets passed straight through to the program's own CommandLine parser instead of being stripped as a separator. Verified directly against this build.)

Valid --scheme values: baseline, gebpdcm, gebpdcm_geo, gebpdcm_ltpm. Prints per-tick safe/threshold/congested/migration counts live, then a final metrics summary, and writes data/ns3_<scheme>_speed<N>.csv into ~/networks_project_v1_ns3/data/ by default (pass --outDir=<path> to override).

CAUTION: if <speed> matches one of the sweep's real speeds (10/20/30/40/50), this OVERWRITES that scheme/speed's saved result. Use a speed outside that set (e.g. --speed=99) for a quick one-off test without disturbing the saved sweep data, or re-run ns3/run_sweep.sh afterwards to restore it.

2.4 Demo animation (Python model, not NS-3 -- see Section 4)

```
cd ~/networks_project_v1_ns3
python3 demo.py
```

3. Running v2 (Pure Python)

v2's engine is src/simulator.py -- an aggregate, per-node model. This runs in seconds, no NS-3 involved after the topology file already exists.

3.1 Run everything

```
cd ~/networks_project_v2_python
python3 run_all.py       # ~5-10 seconds -- all 4 schemes x 5 speeds + packet-size sweep
python3 plot_results.py  # writes the 5 PNGs + results/comparison_table.csv
```

3.2 Optional: run one scheme by hand

```
cd ~/networks_project_v2_python
python3 src/simulator.py
```

Prints one metrics dict for scheme 'gebpdcm' at the default speed (10 m/s) and packet size (1Kb) -- a quick sanity check that the Python engine still runs end-to-end.

3.3 Demo animation

```
cd ~/networks_project_v2_python
python3 demo.py
```

Optional flags: --scheme gebpdcm_geo (or any of the 4 scheme names) to demo a different scheme; --speed 30 to change mobility. Writes results/demo.gif -- takes about a minute.

4. The Live Demo (demo.py)

demo.py is identical in both v1 and v2 -- it's a standalone, animated matplotlib visualisation of the Python aggregate model (src/simulator.py), independent of which version's NS-3/Python split you're demoing otherwise. It exists purely as a visual aid, and is explicitly the lowest-priority deliverable in the project spec -- safe to skip if you're short on time.

What it shows

- 100 nodes plotted at their real (or seeded) positions, moving under random-waypoint mobility.
- Each node coloured green (safe) / yellow (at-threshold) / red (congested) live, tick by tick.
- Black arrows drawn whenever a migration happens, from the congested source to its chosen target.
- A live counter of delivered vs dropped packets in a corner of the plot.

```
python3 demo.py --scheme gebpdcm_ltpm --speed 30
```

5. What to Show an Examiner

5.1 For v1 (real NS-3)

- Open ns3/gebpdcm.cc's header comment -- explains the HR/DSSS bug found and fixed, and why migration has no direct NS-3 equivalent so had to be built as a static-route override.
- Walk through GeoSelect()/GeoFitness() -- the attack/cruise loop the paper's title claims but never builds, now running on real neighbour positions.
- Walk through UpdateTrust()/TxTraceCallback()/DropTraceCallback() -- trust fed by real per-node packet outcomes.
- Run ./ns3/run_sweep.sh (or just point at the already-saved data/ns3_*.csv), then python3 run_all.py && python3 plot_results.py, and open the 5 PNGs -- point out baseline beating all three congestion-controlled schemes, and be ready to explain why (Lab Report Section 7.4).

5.2 For v2 (Python)

- Open src/status.py, src/migrate.py -- point at the docstrings, each cites its equation number and states our interpretation where the paper's own equation was dimensionally inconsistent.
- Open src/geo.py -- walk through the attack/cruise vector loop.
- Open src/ltpm.py -- trust EWMA + linear forecast; explain why no ML model was used.
- Run python3 run_all.py && python3 plot_results.py live, then open the 5 PNGs -- point out LTPM's curve consistently on top.
- Play results/demo.gif, or run python3 demo.py live if there's time, for the visual animated version.

5.3 The single most important thing to lead with

If you only have time to make one point: run BOTH versions' comparison tables side by side and explain WHY they disagree (Lab Report Section 7.4) -- v2 models capacity as independent per-node buckets where migration always helps by construction; v1's real network shares one contended radio channel where rerouting adds airtime rather than relieving it. This single comparison is the most substantive finding in the whole project and demonstrates genuine understanding, not just working code.

6. Troubleshooting

Symptom	Likely cause / fix
plot_results.py fails with a missing-file error	run_all.py must be run first in that same folder -- it writes results/speed_sweep.csv, which plot_results.py reads.
ns3/run_sweep.sh: command not found / permission denied	chmod +x ns3/run_sweep.sh, then re-run.
./ns3 run "gebpdcm -- --scheme=..." fails with "Invalid command-line argument: --"	Drop the "--" -- the correct syntax on this ns-3 version is ./ns3 run "gebpdcm --scheme=... --speed=..." (target and args in one string, space-separated, no "--"). See Section 2.3.
A one-off ./ns3 run test silently changed the saved sweep results	You ran it with --speed equal to one of the 5 real sweep speeds, which overwrote that CSV. Re-run ns3/run_sweep.sh to regenerate the full, correct sweep, or always use a throwaway speed (e.g. 99) for one-off tests.
gebpdcm's output CSVs land in the wrong folder / ~/networks_project/data instead of ~/networks_project_v1_ns3/data	The binary defaults to $HOME/networks_project_v1_ns3/data/ -- pass --outDir=<path> explicitly if you've moved the project elsewhere.
v1's NS-3 sweep produces 0% PDR again	Check ns3/gebpdcm.cc still has DataMode/ControlMode/NonUnicastMode set to DsssRate2Mbps, not DsssRate11Mbps -- see the file header and Study Guide Section 9.5.
python3: ModuleNotFoundError: numpy/pandas/matplotlib	pip3 install numpy pandas matplotlib (add --break-system-packages if WSL's Python blocks a plain pip install).
Results look identical to a previous run after editing gebpdcm.cc	Force a rebuild: cd ~/ns-allinone-3.42/ns-3.42 && ./ns3 build gebpdcm, and confirm the binary's timestamp updated before re-running the sweep.